COMP 361
Assignment 2 - Specifications
Due: October 30, 2026, on myCourses at 23:55

TA Office Hours during the week of Oct 26 to November 2, 2026

Introduction

Each assignment in this course has two sections: Individual Exercises and Team Exercises. Complete the individual exercises independently. Complete the team exercises collaboratively with your team.

This assignment is marked out of 40 points.

Individual Exercises

Do this section on your own.

The exercises in this section asks you to complete the following activities:

- An effort estimation problem using an Activity Graph and a Gantt Chart
- Practice with UML diagrams: UML Activity Diagram and System Flowchart Diagram

Use UMLet to create these artifacts.

To keep the stories easy to remember, we will continue with the same use case story we saw in assignment #1. I provide the story again for your convenience here:

For the following Story:

> The manufacturing company, ABC Industries, produces parts to order for its customers. Each to-order part is called a Job. Customers of ABC Industries call the ABC Sales Rep to order the part they want to build. The Sales Rep enters the order into the company database as a Job. Each Job has an ID number (Job number), a title, a text description, a link to the assembly instructions, and a list of parts. The people who build the parts (handle the Job) are called the Floor Staff. When an individual Floor Staff employee has nothing to do, they go to the Punch-in/out computer to get a Job. They see a list of available jobs (i.e., jobs no one is working on), also known as the Jobs List. From that list, they select a job and enter their employee number to assign the work to themselves. At this point, the system checks whether the employee is already punched into another job. If so, the system punches them out from the previous job and punches them in for the new job. Another table in the database records how much time each Floor Staff employee has worked on a job. If the employee has worked on the job multiple times and/or multiple days, the system keeps a record for each work event. This event contains the date, start and end time, job number, and employee number. To punch out means the computer finds the current open record in the database and updates the end time. To punch in means appending a new record to the

database with the date, job number, employee number, and start time (the end time is left blank). In addition to punching in/out of jobs from the user interface, the Floor Staff employee can also enter a date range to view all the Jobs they worked on (the Jobs Completed Report). The report is divided into days. For each day, it lists the jobs the employee worked on. At the end of the day, it displays the total hours and minutes worked. At the end of the report, it displays the total hours and minutes worked over the entire period.

To summarize, the user of this program (the Punch-in/out program) can select four actions from the computer: punch-in, punch-out, report, and exit program. The punch-in operation will automatically punch out the employee if they are already punched in on another job. The punch-out operation only punches the employee out of a job. The employee should not exit the program, so it should prompt, "Are you sure?" If the user forgot to punch out at the end of the day, the program automatically punches everyone who has not already punched out at 5:30 PM.

Your job is to develop the most well-thought-out, optimal, and clean solution. Note that it is possible the story provides more information than is necessary for a given artifact. Be sure to include only the information that is appropriate for the given artifact (never too much, never too little).

EXERCISE #1 – Effort Estimation [12 points]

Assume there are 3 programmers in your team and assume you are not one of the programmers. That means you only have three people. Draw optimal diagrams for the Story. Your three developers' names are Dev1, Dev2, and Dev3.

Submit the following Artifacts for this exercise:

- The optimal Activity Graph for our Story assuming three developers.
- How many days will it take to complete the program according to the Activity Graph?
- What are the total hours of work that need to be paid to the developers?
- Assuming the work starts January 1st, 2027, draw the Gantt Chart including weekends, holidays, and buffer days. Make sure to indicate when holidays and buffer days have been allocated (use a different colour or a different label).
- On what date will the project end according to the Gantt Chart?
- How many extra days are allocated in the Gantt Chart? Specify weekend days total, holiday days total, and buffer days total as a table, with a grand total.

What To Submit

Submit the following to the myCourses Assignment tool, placing your work in the Assignment 2 box:

- A single PDF called A2Q1 with all the diagrams and answers.

Format your submissions so that they are easy to read.

Rubric

- Activity Graph
  - 0-1 = Syntax
  - 0-1 = Optimality
  - 0-1 = Correctness
  - 0-1 = Completeness
- Days to complete (Activity Graph)
  - 0-1 = Computed correctly
- Total work hours
  - 0-1 = Computed correctly
- Gantt Chart
  - 0-1 = Syntax
  - 0-1 = Optimality
  - 0-1 = Correctness
  - 0-1 = Completeness
- End date (Gantt Chart)
  - 0-1 = Computed correctly
- Extra days
  - 0-0.5 = Computed correctly
  - 0-0.5 = Provided a table

EXERCISE #2 – UML Flow Diagrams [9 points]

In the Story, the punch-in activity is complex because it includes a punching out operation. This is a human to machine activity that can trigger an automatic punching-out event. This does not include the end-of-day automatic punch-out at 5:30 PM. In this question, you are being asked to do the following:

- Create a System Flowchart Diagram for this punch-out operation.
- Redo the punch-out operation using an Activity Diagram.
- Lastly, comment on which drawing technique is more useful in this situation, and why.

What To Submit

Submit the following to the myCourses Assignment tool, placing your work in the Assignment 2 box:

- A single PDF called A2Q2 with all the diagrams and answers.

Format your submissions so that they are easy to read.

Rubric

- System Flowchart Diagram
  - 0-1 = Syntax

    - 0-1 = Optimality
    - 0-1 = Correctness
    - 0-1 = Completeness
- UML Activity Diagram
    - 0-1 = Syntax
    - 0-1 = Optimality
    - 0-1 = Correctness
    - 0-1 = Completeness
- Which diagram technique is best
    - 0-1 = Completeness

Team Exercises

Do this section with your team members.

EXERCISE #4 – Requirements Phase **[19 points]**

For this exercise, you will need to do the following:

1. Select your project theme.
2. Decide on the program you will build, by:
    a. Giving your program a name, and
    b. Provide a story that describes your application program in layperson terms.
3. Describe your project from the user's perspective by:
    a. Providing a single high-level Use-Case Diagram,
    b. Storyboard, and
    c. Wireframes of the most important user interfaces.
4. Tentatively, define your team software development strategy (you can change your mind later), by:
    a. What team type will you use, and why?
    b. What roles will each team member have, and why?
    c. What software evolution strategy will the team use, and why?
    d. What software building strategy will the team use, and why?

NOTE #1 to UBISOFT Teams: Learn Unity and start deciding on which basic assets and algorithms to build. Create vanilla code that you can port into the competition theme once it is announced. You will need to catch up with the other schools. This is the best way to do that. Please contact me for more information or additional guidance. Once the teams have been formed with the Games Club, I will start making formal meetings with those teams.

NOTE #2 about **Scrum Journal**: This is a friendly reminder that you should be starting your private scrum journals now. You should review the rules for these journals in the project description.

What To Submit

- Create a single PDF file called A2Team.PDF with the answers to the above team questions (Exercise 4 items 1-4).
- Only the team contact person will submit this document. Please indicate who is the team contact person on the first page (in case more than one person submitted the PDF in error).
- Make sure to include everybody's name on the first page of the document so that the TA can properly register the grade. Everyone in the team receives the same grade for this portion of the assignment.
- I suggest dividing the work between the team members (to avoid Hero).
- It is required that the team meets together to discuss the solution to these questions.

Rubric

- Project Theme
  - 0-1 = Correctly selected from project options and used correctly in Story.
- Program Name
  - 0/1 = 0 if missing, 1 if present.
- Program Story
  - 0-1 = Minimum one paragraph long and must agree with the selected Theme.
- Use Case Diagram
  - 0-1 = Syntax
  - 0-1 = Optimality
  - 0-1 = Correctness
  - 0-1 = Completeness
- Story Board
  - 0-1 = Syntax
  - 0-1 = Optimality
  - 0-1 = Correctness
  - 0-1 = Completeness
- Wireframes (not mockups)
  - 0-1 = Syntax
  - 0-1 = Optimality
  - 0-1 = Correctness
  - 0-1 = Completeness
- Team type and why
  - 0-1 = Correctly described
- Team member roles assignment and why
  - 0-1 = Correctly described and each person is given a role
- Software Evolution Strategy and why
  - 0-1 = Correctly described
- Software Building Strategy and why
  - 0-1 = Correctly described

Definitions:

- Syntax (syntax correctness)
    - Drawings and writing follow the standards presented in class for the drawing and written work. For example, use-case diagrams strictly follow the drawing style presented in class and not the internet.
- Professional (in presentation)
    - Drawings and writings are not only correct, but they are also presented professionally. For example, no drawings are created by hand. All writing is free from grammatical and spelling errors.
- Quality (of the solution)
    - This relates to the student attempting to find a solution that incorporates all of these elements: execution optimality, a simple algorithm, good use of design principles (like design patterns), and a fast running time. In terms of drawing or writing, the solution image fits within a page or less, and the writing is neither overly wordy nor curt.
- Appropriate
    - Appropriateness refers to the fact that the elements that form the solution would be used in the real-life implementation.
    - The student correctly identified the algorithms and design patterns needed for every case.
- Correct
    - Is the student's solution correct?
- Complete (completeness)
    - Did the student fully answer the question, or did they leave elements out of their solution?
    - A solution may have edge cases or additional features that also need description.
- Graceful (graceful execution)
    - The student's solution handles errors and run-time issues so that the program would not crash or hang, alternatively prompting the user.
- BS
    - At times, students will use a lot of language to answer a question, hoping that their uncertainty about the correct response will be hidden within the volume of technical language as they dance around the solution.
    - Other forms of BS include providing multiple solutions to a question in the hopes of receiving partial grades.